

Kansas Lava to Simulate Circuits



SHAKTHI KANNAN

Kansas Lava is a Haskell library that provides abstractions and powerful combinations to describe and simulate hardware circuits. Hardware descriptions are strongly typed and provide a means to describe hardware in a clear manner and express values that flow between components. You can describe both sequential and combinatorial circuits using Kansas Lava. To install the same on Fedora 20 (x86_64), follow the steps described below:

Download `petersen/cabal-install` repo from <https://copr.fedoraproject.org/coprs/petersen/cabal-install/>. If you do not have `wget` installed on your system, install it first using the following command:

```
# yum install wget
```

You can then run the following from `/etc/yum.repos.d` directory:

```
$ wget https://copr.fedoraproject.org/
coprs/petersen/cabal-install/repo/
fedora-20-i386/petersen-cabal-install-
fedora-20-i386.repo
```

Install `cabal-install` and `Git`.

```
# yum install cabal-install git
```

Download `petersen/ghc-7.8` repo from <https://copr.fedoraproject.org/coprs/petersen/ghc-7.8/> to `/etc/yum.repos.d` using:

```
# wget https://copr.fedoraproject.org/
coprs/petersen/ghc-7.8/repo/fedora-
rawhide-i386/petersen-ghc-7.8-fedora-
rawhide-i386.repo
```

Install Glasgow Haskell compiler (GHC) using:

```
# yum install ghc
```

Update the `cabal` local package list and install sized types using:

```
$ cabal update
$ cabal install sized-types
```

Obtain the latest Kansas Lava code from GitHub using:

```
$ git clone https://github.com/ku-fpg/
kansas-lava.git
```

Enter the following into the checked out Kansas Lava directory, compile and install the same as follows:

```
$ cabal install --only-dependencies
$ cabal build
$ cabal install
```

The tool can help verify the behaviour of a circuit in simulation, generate VHDL for synthesis and also validate whether the generated VHDL is the same as the simulated circuit.

A simple code for the half adder combinatorial circuit is given below:

```
import Language.KansasLava

halfAdd :: Signal i Bool -> Signal i
Bool -> (Signal i Bool, Signal i Bool)
halfAdd a b = (carry, sum)
  where carry = a `and2` b
        sum = a `xor2` b
```

Execute the above example using Glasgow Haskell compiler (GHC):

```
$ ghci halfAdd.hs
ghci> halfAdd low high
(low,high)
ghci> halfAdd high high
(high,low)
```

Kansas Lava implements the idea of an undefined `Signal`, where binary functions like `and2`, `or2` and `xor2` are short circuited. With unknown values, these function as hardware gates.

```
ghci> :m + Language.KansasLava

ghci> undefinedS `or2` low
?
ghci> undefinedS `or2` high
high
ghci> undefinedS `and2` low
low
ghci> undefinedS `and2` high
?
ghci> undefinedS `xor2` low
?
ghci> undefinedS `xor2` high
?
```

`mux` is a function that can be used to select an input. With low in-

put, it picks the first argument in the tuple, and with high input, it picks the second argument in the tuple.

```
ghci> :m + Language.KansasLava

ghci> mux low (0,1) :: Signal c Int
0
ghci> mux high (0,1) :: Signal c Int
1
```

Kansas Lava provides a number of instance declarations and overloads for the signal module.

```
instance (Num a, Rep a) => Num (Signal
i a)

instance (Bounded a, Rep a) => Bounded
(Signal i a)

instance (Show a, Bits a, Rep a) => Bits
(Signal i a)
```

The overloading enables you to use Haskell expressions with `Signal` `c` and can be helpful in describing hardware signals.

```
ghci> :m + Language.KansasLava

ghci> 2 :: Signal c Int
2
ghci> 2 * 4 :: Signal c Int
8

ghci> :m + Data.Bits
ghci> 8 `shiftR` 2 :: Signal c Int
2
```

The clock class can change a signal with time and is used for sequential values. It provides two important functions: register and delay. The register function accepts an initial value and an input sequence as arguments, while delay function is similar to register function with the initial value undefined. Register function behaves like D edge-triggered flip-flop with an internal clock. `toS` function converts a set of values into a signal. For example,

```
ghci> :m + Language.KansasLava
```